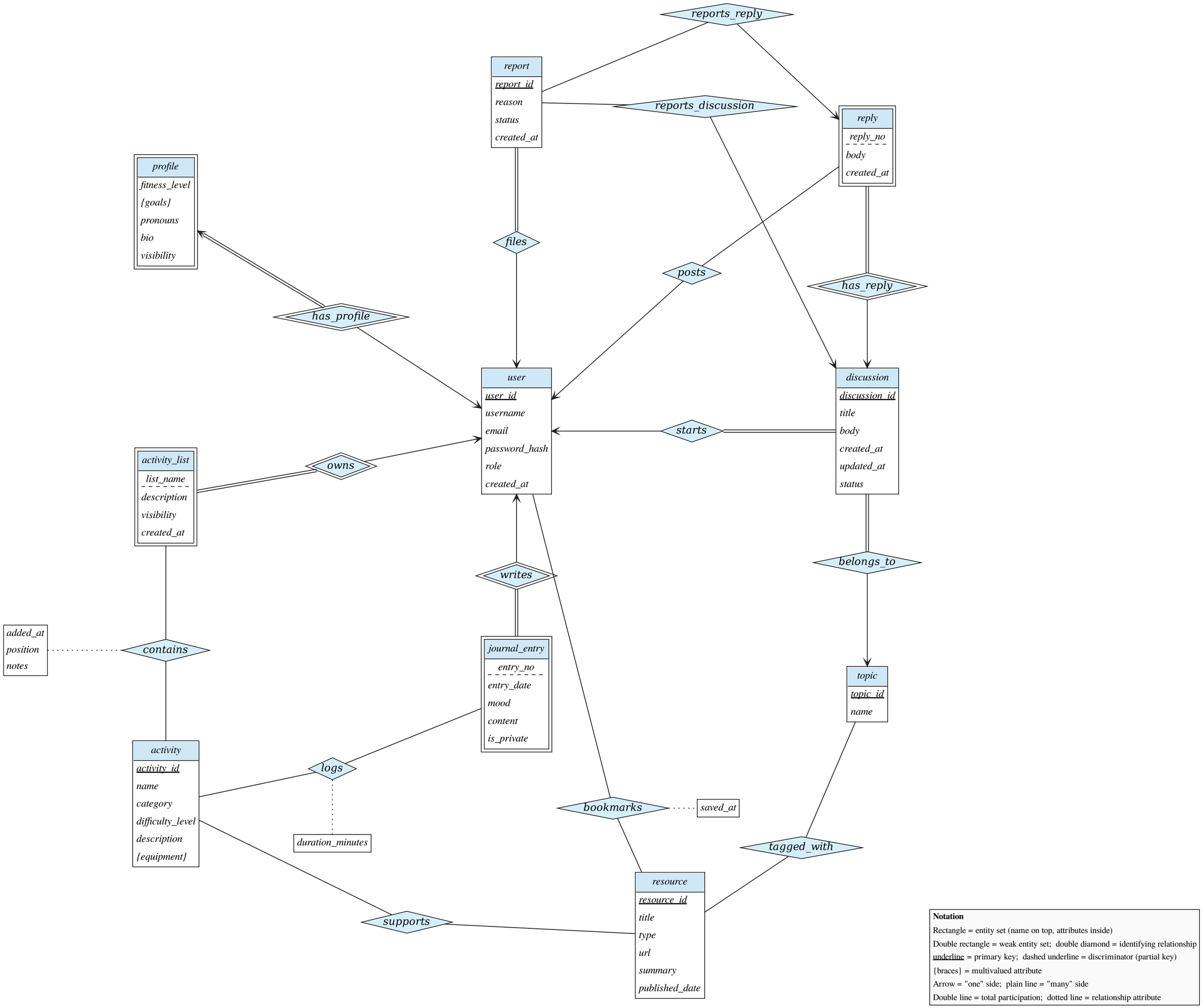
Qelp: ER Diagram



Notation

Rectangle = entity set (name on top, attributes inside)

Double rectangle = weak entity set; double diamond = identifying relationship

underline = primary key; dashed underline = discriminator (partial key)

{braces} = multivalued attribute

Arrow = "one" side; plain line = "many" side

Double line = total participation; dotted line = relationship attribute

Qelp ER Diagram: My Explanation

CS-GY 6083 | Shivam Singh. The diagram is on page 1. I drew it in the same notation as the course slides: entity sets are boxes with their attributes inside, relationships are diamonds.

What I modeled

user is the center because everything private in Qelp belongs to a user. Each user has one **profile**, owns many **activity_lists**, and writes many **journal_entries**. Lists and journal entries both point to shared **activities** (yoga, walking, strength training). For the community part, users start **discussions** under a **topic** and write **replies**. **resources** share topics with discussions, users can bookmark them, and they can support activities. **report** is for moderation.

How to read the notation

- Plain rectangle = strong entity set. **Double rectangle** = weak entity set (it needs an owner to exist): profile, activity_list, journal_entry and reply.
- Diamond = relationship. **Double diamond** = identifying relationship (the one that connects a weak entity to its owner).
- Underlined attribute = primary key. Dashed underline = discriminator, the partial key of a weak entity. {Braces} = multivalued attribute (goals, equipment).
- **Arrow** points at the "one" side. A plain line is the "many" side. **Double line** = total participation (every one of them must take part). A dotted line connects a relationship to its own attributes.

My entity sets

Entity set	Kind	Key	What it is
user	Strong	user_id	Account. The *role* attribute (member, moderator, admin) is my user type.
profile	Weak	owner's user_id	Fitness level, goals, pronouns, bio, visibility. One per user. Pronouns are optional.
activity	Strong	activity_id	Shared catalog of movements. Equipment is multivalued.
activity_list	Weak	list_name	A user's saved list. Identified by its owner plus list_name.
journal_entry	Weak	entry_no	Private wellness record with mood and content. Identified by owner plus entry_no.
discussion	Strong	discussion_id	A community thread with a title, body and status.
reply	Weak	reply_no	A response. It only exists inside its discussion.
topic	Strong	topic_id	Shared label for discussions and resources.
resource	Strong	resource_id	Educational article or material.
report	Strong	report_id	Moderation report with a reason and status.

My relationships

Relationship	Between	Cardinality	Notes
has_profile	user, profile	1 : 1	Identifying. Profile is total.
owns	user, activity_list	1 : many	Identifying. List is total.

Relationship	Between	Cardinality	Notes
contains	activity_list, activity	many : many	Own attributes: added_at, position, notes.
writes	user, journal_entry	1 : many	Identifying. Entry is total.
logs	journal_entry, activity	many : many	Own attribute: duration_minutes. Optional.
starts	user, discussion	1 : many	Every discussion has an author (total).
belongs_to	discussion, topic	many : 1	Every discussion has a topic (total).
has_reply	discussion, reply	1 : many	Identifying. Reply is total.
posts	user, reply	1 : many	Who wrote the reply. Not identifying.
tagged_with	resource, topic	many : many	A resource can have several topics.
bookmarks	user, resource	many : many	Own attribute: saved_at.
supports	resource, activity	many : many	Which resources explain which activities.
files	user, report	1 : many	Who filed the report (total).
reports_discussion, reports_reply	report, discussion / reply	many : 1 each	A report targets exactly one of the two. The diagram can't show that, so I enforce it with an application check.

Why I made these choices

- **User type is the role attribute, not a separate entity.** Members, moderators and admins have the same attributes and relationships, so one attribute with permitted values is enough and it becomes schema validation in MongoDB. If a type ever needs its own attributes, I would switch to an ISA hierarchy.
- **Profile, activity_list, journal_entry and reply are weak** because none of them make sense without their owner. Deleting a user should delete their lists, entries and profile.
- **journal_entry uses entry_no as its discriminator, not entry_date,** because I can write more than one entry in a day. entry_date is a normal attribute that the date-range query uses.
- **Date range and sorting are not entities.** They are query conditions, so they show up in my indexes (for example user + entry_date, and topic + created_at), not in the diagram.
- **Privacy is part of the design.** Pronouns are optional, profile and list fields have a visibility setting, and journal entries have an is_private flag.
- **report is optional.** My proposal says moderation depends on the time I have, so it is the first thing I cut.

How this becomes MongoDB

Diagram	Collection design	Why
user + profile	**users**, profile embedded	1:1 and always read together.
activity	**activities**, equipment as an array	Shared catalog. Indexes on category, difficulty, equipment.
activity_list + contains	**activity_lists**, items embedded as {activity_id, position, notes, added_at}	Read as a whole and small. I reference the activity instead of copying it so nothing goes stale.
journal_entry + logs	**journal_entries**, referenced by user_id	Grows without limit. Index on user_id + entry_date.
discussion, reply	**discussions** and **replies**, referenced	Indexes on topic + created_at and on discussion_id.

Diagram	Collection design	Why
resource + tagged_with	**resources**, topics as an array	Index on the topic array.
topic, report, bookmarks	Optional extra collections or embedded fields	My proposal lists seven collections, so I decide these based on time.